软件测试重点

📚 课件重点章节目录

🔥 核心重点章节（必考）

章节	重点内容	考试频率
第1章 软件测试基础	软件测试的定义、目的和意义；软件测试的原则（9条原则）；软件测试的流程；软件测试与软件开发的关系；软件测试的关键问题	★★★★★
第3章 黑盒测试方法（含3.2、3.4）	等价类划分法（有效等价类、无效等价类）；边界值分析法；因果图法；错误推测法；正交表法；场景法、功能图法	★★★★★
第4章 白盒测试方法	逻辑覆盖（语句覆盖、判定覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖、路径覆盖）；基本路径测试；控制流图绘制；圈复杂度计算；独立路径导出	★★★★★
第8章 单元测试	单元测试的原则（CORRECT原则、A-trip原则）；单元测试的主要测试问题；单元测试的重要性及目的	★★★★
第9章 集成测试	集成测试的方法（非增式测试、增式测试）；自顶向下、自底向上测试；集成测试与单元测试、系统测试的区别	★★★★
第7章 系统测试技术	系统测试的内容；系统测试与用户测试的区别；功能测试、性能测试、安全测试、兼容性测试；恢复测试、强度测试、正确性测试	★★★★

⭐ 重要章节（常考）

章节	重点内容	考试频率
第2章 软件测试策略	V模型、W模型、H模型、X模型；软件测试阶段（单元测试、集成测试、系统测试、验收测试）	★★★★
第5章 软件测试的过程管理	测试计划制定；测试用例设计原则和要素；测试停止标准；测试需求评审	★★★
第6章 软件测试的度量	代码覆盖率计算；功能覆盖率计算；数据库覆盖率计算；测试度量原则	★★★
第10章 数据库测试	数据库应用软件与DBMS的区别；数据库模式验证；DBMS事务特性测试；DBMS性能测试	★★★

📖 一般章节（了解即可）

章节	重点内容	考试频率
第9章 第三方测试	第三方测试的定义、意义、模式；第三方测试的范围和过程	★★
第11章 智能软件测试技术	智能搜索算法（深度优先、广度优先、A*算法）；遗传算法；变异测试	★★
第12章 公有云测试质量评估	云测试的两层含义；云安全技术；云计算安全与传统软件安全的区别	★
第13章 软件测试的拓展与提高	测试驱动开发（TDD）；API接口测试；移动应用测试；A/B测试；探索性测试	★★

📊 章节优先级总结

优先级	章节	考试频率	建议复习时间
P0（必考）	第1章、第3章、第4章	★★★★★	60%
P1（常考）	第8章、第9章、第7章、第2章	★★★★	25%
P2（重要）	第5章、第6章、第10章	★★★	10%
P3（了解）	第9章、第11章、第12章、第13章	★★	5%

🔥 第1章 软件测试基础

1.1 什么是软件测试?

- 验证软件功能是否符合需求的过程
- 发现软件缺陷的活动
- 评估软件质量的手段

1.2 软件测试的目的和意义是什么?

答案:

目的:

1. 以最少的人力、物力、时间**找出软件中潜在的各种错误和缺陷**，通过修正错误和缺陷提高软件质量，**回避潜在的软件错误和缺陷给软件造成的商业风险**。
2. 通过分析测试过程中发现的问题可以帮助**发现当前开发工作所采用的软件过程的缺陷**，以便进行软件过程改进；同时通过对测试结果的分析整理，可**修正软件开发规则**，并为软件可靠性分析提供相关的依据。
3. 评价程序或系统的属性，对软件**质量进行度量和评估**，以验证软件的质量满足用户的需求，为用户选择、接受软件提供有力的依据。

意义:

- 保证软件质量
- 降低商业风险
- 改进开发过程
- 提供质量评估依据

1.3 软件测试应遵循哪些重要的原则?

- 尽早测试
- 测试用例的完备性
  - 重视无效数据和非预期使用习惯的测试
  - 用例要定期评审
- 第三方测试
- 避免开发人员测试自己的程序
- 彻底检查每个测试结果
- 软件测试的经济性原则

1.4 软件测试涉及哪几个关键问题?

1. 测试对象和范围
2. 测试时机和策略
3. 测试资源分配
4. 测试完成标准

1.5 软件测试的流程是什么？

1. 测试计划制定
2. 测试用例设计
3. 测试环境搭建
4. 测试执行
5. 缺陷管理
6. 测试报告编写

与软件开发各阶段的关系：

- 需求分析 —— 确认测试
- 设计阶段 —— 集成测试
- 编程 —— 单元测试

1.6 软件测试与软件开发有何关系？

- 测试贯穿整个开发生命周期
- 测试与开发并行进行
- 测试反馈促进开发改进
- 测试验证开发成果

软件开发是一个自顶向下、逐步细化的过程：

- 软件计划阶段定义软件作用域
- 软件需求阶段分析建立了软件信息域、功能和性能需求、约束等
- 软件设计把设计用某种程序设计语言转换成程序代码

测试过程是依相反顺序自底向上，逐步集成的过程：

- 对每个程序模块进行单元测试，消除程序模块内部逻辑和功能上的错误和缺陷
- 对照软件设计进行集成测试，检测和排除子系统或系统结构上的错误
- 对照需求，进行确认测试
- 最后从系统全体出发，运行系统，看是否能够满足要求

1.7 简述软件测试的复杂性和经济性。

复杂性：

- 软件规模和复杂度不断增加
- 测试场景组合爆炸
- 测试环境依赖复杂

经济性：

- 测试成本与质量平衡
- 投入产出比考量
- 测试资源优化配置

1.8 软件开发模型在软件开发过程中起到什么作用？没有它可以吗？

- 规范开发流程
- 指导项目管理
- 提高开发效率
- 降低项目风险

不能没有开发模型，因为缺乏规范会导致混乱和低效

1.9 软件测试中检测到的错误都是由编码错误引起的吗,还有哪些

不全是编码错误

- 需求分析错误
- 设计缺陷
- 文档歧义
- 环境配置问题
- 集成问题

1.10 静态测试和动态测试分别是什么？

静态测试（static testing）：

- 指不实际运行被测软件，而只是静态地检查程序代码、界面或文档中可能存在的错误过程。
- **代码测试**：主要测试代码是否符合相应的标准和规范。
- 界面测试：主要测试软件的实际界面与需求中的说明是否相符。
- **文档测试**：主要测试用户手册和需求说明是否真正符合用户的实际需求。
- **设计评审**
- **工具**：Logiscope、Telelogic等，可以用作Java/C++等规范测试。

动态测试（dynamic testing）：

- 指实际运行被测程序，输入相应的测试数据，检查实际输出结果和预期结果是否相符的过程。
- 动态测试方法为结构和正确性测试。
- **功能验证**
- **性能测试**
- **集成测试**
- **工具**：Robot、QTP等。

1.11 黑盒测试和白盒测试的区别是什么？

可以同时使用，互补性强

黑盒测试（数据驱动测试）：

1. 不考虑内部结构
2. 基于功能规范

3. 测试数据选择

- 完全**不考虑程序内部结构和内部特性，注重于测试软件的功能需求。**
- 把被测程序看成是一个打不开的黑盒，测试人员在不考虑程序内部结构和内部特性的情况下，只根据需求规格说明书，设计测试用例，**检查程序的功能是否按照规范说明的规定正确执行**。
- 不需要了解程序内部结构，测试人员不需要具备编程能力，**测试成本较低**，可以从用户角度测试。
- 缺点：无法检查程序的内部逻辑，可能遗漏代码级别的错误，测试覆盖率难以保证。

白盒测试（结构测试或逻辑驱动测试）：

1. 基于代码结构
2. 关注逻辑路径
3. 代码覆盖率

- 把盒子打开，去研究里面的源代码和程序结构。
- 该方法将被测对象看作一个打开的盒子，允许人们检查其内部结构。测试人员根据程序内部的结构特性，设计和选择测试用例，检测程序的每条路径是否都按照预定的要求正确执行。
- 可以检查程序的内部逻辑，可以发现代码级别的错误，可以评估代码覆盖率。
- 缺点：需要了解程序内部结构，测试人员需要具备编程能力，测试成本较高。

关系：

- 软件公司往往采用两种测试相结合的方式
- 整体功能和性能进行黑盒测试
- 源代码采用白盒测试

1.12 什么是白盒测试和黑盒测试的优缺点？

白盒测试优缺点：

- **优点**：可以检查程序的内部逻辑、可以发现代码级别的错误、可以评估代码覆盖率
- **缺点**：需要了解程序内部结构、测试人员需要具备编程能力、测试成本较高

黑盒测试优缺点：

- **优点**：不需要了解程序内部结构、测试人员不需要具备编程能力、测试成本较低、可以从用户角度测试
- **缺点**：无法检查程序的内部逻辑、可能遗漏代码级别的错误、测试覆盖率难以保证

1.13 请说明单元测试、集成测试和系统测试的区别。

单元测试：

- 测试最小单元
- 开发人员执行

集成测试：

- 测试模块间接口
- 关注组件交互

系统测试：

- 测试整体功能
- 端到端验证

⭐ 第2章 软件测试策略

2.1 软件测试包括哪几个阶段？

1. 单元测试
2. 集成测试
3. 系统测试
4. 验收测试

与软件开发各阶段的关系：

- 单元测试：对源程序中每一个程序单元进行测试，检查各个模块是否正确实现规定的功能，从而发现模块在编码中或算法中的错误。该阶段涉及编码和详细设计文档。
- 集成测试：是为了检查与设计相关的软件体系结构的有关问题，也就是检查概要设计是否合理有效。
- 确认测试：主要是检查已实现的软件是否满足需求规格说明书中确定了的各种需求。分Alpha测试和Beta测试。
- 系统测试：是把已确认的软件与其他系统元素（如硬件、其他支持软件、数据、人工等）结合在一起进行测试。以确定软件是否可以支付使用。

2.2 什么是V模型？它的阶段和特点是什么？

V模型定义：
V模型是一种软件开发生命周期模型，它将开发过程和测试过程对应起来，形成了一个V字形的结构。

V模型的阶段和特点：

- **需求分析** —— 确认测试
- **设计阶段** —— 集成测试
- **编程** —— 单元测试

特点：

- 测试活动与开发活动并行进行
- 每个开发阶段都有对应的测试阶段
- 强调测试的早期介入
- 测试用例基于需求和设计文档

2.3 常用的软件测试模型有哪些？

答案：

1. **V模型**
2. **W模型**
3. **H模型**
4. **X模型**
5. **前置测试模型**

🔥 第3章 黑盒测试方

3.1 什么是等价类划分法？如何应用？

定义：
等价类划分法是从大量输入数据划分成若干部分（等价类/子集），然后从每个部分中挑选少量代表性数据作为测试用例，这些代表数据要能反应这个范围内数据的测试结果。

等价类类型：

- **有效等价类**：合法输入范围，对程序的规格说明而言，合理且有意义的输入数据构成的集合
- **无效等价类**：非法输入范围，对程序的规格说明而言，不合理的、无意义的输入数据构成的集合——**无效值的等价类**
- **划分方式**：
  - 根据业务规则划分
  - 基于数据类型特征

3.2 请简述等价类划分法的操作流程

1. 识别输入条件
2. 确定有效等价类
3. 确定无效等价类
4. 为每个等价类编号
5. 设计测试用例
6. 检查测试用例的完整性

示例：年龄输入框

- 有效：0 - 120
- 无效：负数，超大数值

示例：用户名注册系统

- 有效等价类：
  - 长度在6-20之间的合法用户名
  - 以字母开头
  - 仅包含字母、数字和下划线
- 无效等价类：
  - 长度小于6的用户名
  - 长度大于20的用户名
  - 不以字母开头的用户名
  - 包含特殊字符的用户名

测试用例：

1. "abc123"（有效）
2. "abcdef_123456789"（有效）
3. "ab"（无效：长度过短）
4. "abcdefghijklmnopqrst1"（无效：长度过长）
5. "123abc"（无效：不以字母开头）
6. "abc@123"（无效：包含特殊字符）

3.3 弱健壮等价类，强健壮等价类，弱一般等价类，强一般等价类定义

弱一般等价类测试

不考虑无效数据，**测试用例使用每个等价类中的一个值**。它选取的测试用例只需要覆盖到有效等价类，

强一般等价类测试

每个等价类至少要选择一个测试用例。它**不考虑无效等价类**，选取测试用例时，各个有效区间的组合都要覆盖到

弱健壮等价类测试

对于有效输入，使用每个有效等价类的一个值；对于无效输入，**测试用例只使用一个无效值**，其余值都是有效的。

强健壮等价类测试

每个无效等价类和有效等价类的组合都要覆盖到，考虑所有的有效和无效情况，

3.4 对于三角形问题，给出弱健壮等价类测试用例。

弱：

- 三边都是有效值

健壮：

- 包含无效输入
- 零值边长
- 负数边长
- 不能构成三角形的组合

3.5 黑盒测试中，测试人员和程序员应该相互独立。解释其合理性。

- 避免开发思维定式
- 保持客观性
- 模拟用户视角
- 提高测试有效性

3.6 若测试机器学习程序，请设计出一些蜕变关系。

- 数据缩放不变性
- 特征顺序置换
- 预测结果一致性
- 数据增广等价性

3.7 什么是边界值分析法?

定义：
程序的很多错误发生在输入或输出范围的边界上，因此针对边界情况设置测试用例，可以发现不少程序缺陷。

测试边界点及其邻近值

边界是指：

- 有效值范围的临界点
- 常见边界：最大/最小值、临界状态
- 包括：正好在边界上、刚好超过、刚好在边界之下

3.8 什么是因果图法?

定义：
适合输入条件比较多的情况，测试所有的输入条件的排列组合。

image-20260111155756620

image-20260111155853247

image-20260111160033379

示例：

3.9 有一个饮料自动售货机的控制处理软件。若投入 5 角钱的硬币，按下橙汁或啤酒的按钮，则相应的饮料就送出来。若投入 1 元的硬币，同样也是按下橙汁或啤酒的按钮，则相应的饮料会送出来并退还 5 角硬币。试画出因果图，并给出对应的测试用例。

输入条件：

- C1：投入 5 角
- C2：投入 1 元
- C3：按橙汁键
- C4：按啤酒键

输出结果：

- E1：出橙汁
- E2：出啤酒
- E3：退 5 角

测试用例应覆盖各种组合情况

3.10 什么是正交分析法?

正交分析法是从大量试验点中挑出适量的、有代表性的点，利用"正交表"，合理的安排试验对所有变量对的组合进行典型覆盖。

正交表的选择和测试用例设计：

- 根据变量的数量和每个变量的取值数量选择合适的正交表
- 常见正交表：L8(2$^{7)、L9(3}$4)、L16(4^5)
- L表示正交表，后面的数字表示试验次数，括号内的数字表示变量的取值数量和变量数量

3.11 常见的黑盒测试用例设计方法有哪些?

1. **等价类划分法**：从大量数据中划分范围（等价类），然后从每个范围中挑选代表数据，这些代表数据要能反应这个范围内数据的测试结果。
2. **边界值分析法**：程序的很多错误发生在输入或输出范围的边界上，因此针对边界情况设置测试用例，可以发现不少程序缺陷。
3. **错误推测法**：基于经验和直觉推测推测所有可能存在的错误，有针对性的设计测试用例的方法。
4. **因果图法**：适合输入条件比较多的情况，测试所有的输入条件的排列组合。
5. **正交表实验法**：从大量试验点中挑出适量的、有代表性的点，利用"正交表"，合理的安排试验对所有变量对的组合进行典型覆盖。
6. **组合测试法**

🔥 第4章 白盒测试方法

4.1 白盒测试的覆盖准则有哪些?

代码覆盖率：测试覆盖率：用于确定测试所执行到的覆盖项的百分比，其中覆盖项是指作为测试基础的一个入口或属性，如语句、分支、条件等

白盒测试法的覆盖标准有逻辑覆盖、循环覆盖和基本路径测试。其中逻辑覆盖包括语句覆盖、判定覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖和路径覆盖。六种覆盖标准发现错误的能力呈由弱到强的变化。

1. **语句覆盖**：选择足够多的测试用例，使得程序中的每个可执行语句至少执行一次。
2. **判定覆盖（分支覆盖）**：通过执行足够的测试用例，使得程序中的每个判定至少都获得一次"真"值和"假"值，也就是使程序中的每个取"真"分支和取"假"分支至少均经历一次。
3. **条件覆盖**：设计足够多的测试用例，使得程序中每个判定包含的每个条件的可能取值（真/假）都至少满足一次。
4. **条件/判定组合覆盖**：每个判断中的每个条件的可能取值至少满足一次，每个判断至少获得一次"真"和一次"假"。
5. **组合覆盖**：每个判定的所有可能的条件取值都至少出现一次。
6. **路径覆盖**：使程序中每一条可能的路径至少执行一次。

4.2 什么是基本路径测试？如何进行?

定义：
基本路径测试是白盒测试的一种方法，通过控制流图、圈复杂度和独立路径来设计测试用例。

在程序控制流图的基础上，通过分析控制构造的环路复杂性，导出基本可执行的路径集合，从而设计测试用例的方法

步骤：

1. **画出控制流图**
   - 将程序流程转换为控制流图
   - 节点表示程序语句
   - 边表示控制流
2. **计算圈复杂度**
   - 公式：V(G) = 边数 - 节点数 + 2
   - 或：V(G) = 判定节点数 + 1
3. **导出独立路径**
   - 独立路径数量 = 圈复杂度
   - 每条独立路径至少包含一条新的边
4. **设计测试用例**
   - 为每条独立路径设计测试用例
   - 确保覆盖所有独立路径

示例：findMax函数

```
int findMax(int a, int b, int c) {
    int max = a;
    if (b > max) {
        max = b;
    }
    if (c > max) {
        max = c;
    }
    return max;
}
```

控制流图：

- 节点1：max = a
- 节点2：b > max
- 节点3：max = b
- 节点4：c > max
- 节点5：max = c
- 节点6：return max

圈复杂度：
V(G) = 边数 - 节点数 + 2 = 7 - 6 + 2 = 3

独立路径：

1. 1 → 2 → 4 → 6
2. 1 → 2 → 3 → 4 → 6
3. 1 → 2 → 3 → 4 → 5 → 6

测试用例：

1. a=5, b=3, c=2（期望输出：5）
2. a=3, b=5, c=4（期望输出：5）
3. a=3, b=5, c=7（期望输出：7）

4.3 条件覆盖和分支覆盖有什么区别?

答案：

条件覆盖：

- 设计足够多的测试用例，使得程序中每个判定包含的每个条件的可能取值（真/假）都至少满足一次。
- 关注判定中的每个条件是否都取到真值和假值。

分支覆盖（判定覆盖）：

- 通过执行足够的测试用例，使得程序中的每个判定至少都获得一次"真"值和"假"值。
- 关注每个判定的真假分支是否都被执行过。

示例：**insuranceCost函数**

```java
int insuranceCost(int age, boolean isSmoker, boolean hasDisease) {
    int p = 1000;
    if (age > 60 && isSmoker) {
        p *= 2;
    } else if (age > 30 && hasDisease) {
        p *= 1.5;
    } else if (age <= 30) {
        p *= 0.8;
    }
    return p;
}
```

条件覆盖测试用例：

1. age=65, isSmoker=true, hasDisease=false（premium=2000.0）
2. age=35, isSmoker=false, hasDisease=true（premium=1500.0）
3. age=25, isSmoker=false, hasDisease=false（premium=800.0）
4. age=45, isSmoker=false, hasDisease=false（premium=1000.0）

分支覆盖测试用例：

1. age=65, isSmoker=true, hasDisease=false（premium=2000.0）
2. age=35, isSmoker=false, hasDisease=true（premium=1500.0）
3. age=25, isSmoker=false, hasDisease=false（premium=800.0）

4.4 请说明测试覆盖率和测试充分性的关系。

覆盖率类型：

- 代码覆盖
- 功能覆盖
- 场景覆盖

充分性考量：

- 业务重要性
- 风险程度
- 成本效益

关系：

- 高覆盖率不等于充分测试
- 需综合评估

4.5 程序变异的基本思想是什么？—评估测试用例质量

- 在原程序基础上制造缺陷
- 通过测试用例检测变异体
- 评估测试用例质量
- 找出测试薄弱环节

4.6 什么是变异算子?

- 算术运算符替换
- 关系运算符替换
- 逻辑运算符替换
- 常量修改
- 语句删除/插入

4.7 一阶变异和高阶变异有什么区别?

一阶变异：

- 只对程序进行单个变异

- 计算成本低
- 容易分析

高阶变异：

- 同时进行多个变异
- 更接近实际缺陷
- 计算成本高

⭐ 第5章 软件测试的过程管理

软件测试阶段

image-20260111190418476

5.1 什么是软件测试计划？

软件测试计划是指导测试过程的纲领性文件，对测试过程的一个整体上的设计

image-20260111190626815

5.2 测试用例的设计原则和要素是什么？

测试用例概念：
测试用例，英文为TestCase，缩写为TC，指的是在测试执行之前设计的一套详细的测试方案，包括测试环境、测试步骤、测试数据和预期结果。

测试用例元素（组成部分）：

- 测试环境
- 测试步骤
- 测试数据
- 预期结果

测试用例设计原则：

1. 正确性
2. 全面性
3. 整体连贯性
4. 可维护性
5. 测试结果可判定性和可再现性
6. 经济性原则
7. 可追溯性原则

设计方法：

- 等价类划分法
- 边界值分析法
- 基本路径分析法
- 因果图法

测试用例的设计粒度：

1. 复用率
2. 项目进展
3. 使用对象
4. 测试种类

测试用例要素

要素	说明	示例
用例编号	唯一标识	TC_LOGIN_001
用例标题	简洁描述	验证正确用户名密码登录成功
所属模块	功能模块	用户登录模块
优先级	重要程度	高/中/低 或 P0/P1/P2
前置条件	执行前的准备	用户已注册，数据库正常
测试数据	输入数据	用户名：test@qq.com，密码：Test123!
测试步骤	详细操作	见下文详细说明
预期结果	期望输出	登录成功，跳转首页
实际结果	执行后填写	（执行时填写）
测试结果	通过/失败	（执行时填写）
备注	补充说明	仅适用于Web端

5.3 测试停止标准有哪些？

1. 基于覆盖率的标准（定量）
2. 基于缺陷的标准（定量）
3. 基于时间的标准（定性）
4. 基于风险的标准（定性）
5. 基于资源的标准（定性）
6. 综合标准（多维度）

5.4 测试需求评审需要从哪几个方面进行？

1. 需求的完整性
2. 需求的可测试性
3. 需求的一致性
4. 需求的可追溯性
5. 需求的优先级

5.5 请说明测试执行过程中所要做的主要工作。

- 准备测试环境
- 执行测试用例

- 记录测试结果
- 缺陷提交与跟踪
- 测试进度报告

5.6 软件缺陷的生命周期是什么?

发现 → 提交 → 分配 → 修复 → 验证 → 关闭

5.7 软件缺陷的级别有哪些?

1. **致命缺陷**：导致系统崩溃或数据丢失
2. **严重缺陷**：主要功能无法使用
3. **一般缺陷**：影响功能但有替代方案
4. **轻微缺陷**：界面或提示信息问题

⭐ 第6章 软件测试的度量

6.1 代码行覆盖率如何计算？功能覆盖率如何计算？数据库覆盖率如何计算?

代码行覆盖率：

- 已执行行数 / 总代码行数

功能覆盖率：

- 已测试功能点 / 总功能点

数据库覆盖率：

- 已测试的数据库对象 / 总数据库对象

6.2 软件测试的度量

包括对软件测试的产出物的测量，以及测试的 过程的测量。

有如下目的：

- 判断软件测试的有效性。
- 判断软件测试的完整性。
- 判断所测试的软件产品的质量。
- 分析和改进软件测试过程。

6.3 软件测试度量涉及哪几个关键问题?

- 度量指标的选择
- 数据收集方法
- 度量结果分析
- 改进措施制定

6.4 软件测试度量应遵循哪些重要的原则?

- 可度量性
- 成本效益
- 客观性
- 及时性
- 可比性

6.5 软件测试度量是出于什么原因才进行的？是不可或缺的吗?

- 评估测试效果
- 指导测试决策
- 优化测试过程
- 预测项目风险

不可或缺，因为无法度量就无法改进

🔥 第7章 系统测试技术

7.1 自动化测试与测试自动化区别：

自动化测试：

- 具体测试执行的自动化
- 关注单个测试案例

测试自动化：

- 整个测试过程的自动化
- 包括管理、执行、报告等

7.2 自动化测试的优缺点是什么?

优点：

1. 对程序回归测试更方便
2. 建立可靠、重复的测试，减少人为失误，更好地利用资源
3. 增强测试质量和覆盖率
4. 执行手工测试不可能完成的任务

缺点：

1. 不能取代手工测试，自动化测试没有思维，设计的好坏决定了测试质量
2. 发现的问题和缺陷比手工测试要少
3. 不能用于测试周期很短的项目，不能保证100%的测试覆盖率，不能测试不稳定的软件和软件易用性

7.3 Web 系统具有什么特征?

- 分布式架构

- 多层结构
- 跨平台性
- 并发访问
- 安全性要求高

-
  ```
  动态性、异构性、并发性和分布性
  ```

7.4 什么是性能测试?

性能测试是指测试软件是否达到需求规格说明书中规定的各项性能指标，并满足相关的约束和限制。是软件测试的高端领域。

性能测试类型：

- 时间性能
- 空间性能
- 一般性能测试
- 可靠性测试
- 负载测试
- 压力测试

性能测试的基本步骤和流程：

1. 确定测试指标
2. 设计测试场景
3. 准备测试数据和环境
4. 执行测试
5. 监控分析
6. 优化建议

7.5 什么是系统测试?

系统测试指的是将整个软件系统看为一个整体进行测试，包括对功能、性能、以及软件所运行的软硬件环境进行测试。

目前系统测试主要由黑盒测试工程师在系统集成完毕后进行测试，前期主要测试系统的功能是否满足需求，后期主要测试系统运行的性能是否满足需求，以及系统在不同的软硬件环境中的兼容性等。

7.6 系统测试与用户测试的区别是什么？**nonono**

系统测试：

- 指的是将整个软件系统看为一个整体进行测试，包括对功能、性能、以及软件所运行的软硬件环境进行测试。
- 目前系统测试主要由黑盒测试工程师在系统集成完毕后进行测试，前期主要测试系统的功能是否满足需求，后期主要测试系统运行的性能是否满足需求，以及系统在不同的软硬件环境中的兼容性等。

用户测试：

- 属于第三方测试
- 第三方测试是由开发者和用户以外的第三方进行的软件测试，其目的是为了保证测试的客观性。
- 狭义的理解是独立的第三方测试机构，如国家级软件评测中心，各省软件评测中心，有资质的软件评测企业。广义的理解是非本软件的开发人员。如QA部门人员测试、公司内部交叉测试。
- 用户测试：很难进行全面的功能性测试，其他的性能、并发等方面的测试比较困难。

7.7 系统测试的主要内容有哪些？**nonono**

包括对功能、性能、以及软件所运行的软硬件环境进行测试。

目前系统测试主要由黑盒测试工程师在系统集成完毕后进行测试，前期主要测试系统的功能是否满足需求，后期主要测试系统运行的性能是否满足需求，以及系统在不同的软硬件环境中的兼容性等。

7.8 常见的系统测试方法有哪些？**nonono**

1. **恢复测试**
   - 恢复测试指持续超过系统规格负载测试之后，再将负载恢复到规格以内的测试方法，同时，恢复测试还关注导致软件运行失败的各种条件，并验证其恢复过程能否正确执行。在特定情况下，系统需具备容错能力。另外，系统失效必须在规定时间段内被更正，否则将会导致严重的经济损失。
2. **安全测试**
   - 安全测试用来验证系统内部的保护机制，对信息、数据的保护能力，以防止非法侵入。在安全测试中，测试人员扮演试图侵入系统的角色，采用各种办法试图突破防线。因此系统安全设计的准则是要想方设法使侵入系统所需的代价更加昂贵。
3. **压力测试**
   - 压力测试指一段时间内持续超过系统规格的负载进行测试的一种可靠性测试方法。
4. **功能测试**
   - 功能性测试-适合性、准确性、互操作性、安全性、依从性
5. **性能测试**
   - 性能测试类型（时间性能、空间性能、一般性能测试、可靠性测试、负载测试、压力测试）
6. **兼容性测试**
   - 测试软件在不同的硬件平台上、不同的应用软件之间、不同的操作系统中、不同的网络环境中是否可以正常的运行、有无异常的测试过程。
7. **强度测试**
   - 使软件在其设计能力的极限状态下，以及超过此极限下运行，检验软件对异常情况的抵抗能力。
8. **正确性测试**
   - 正确性测试检查软件的功能是否符合规格说明。

🔥 第8章 单元测试

8.1 什么是单元测试?

单元测试又称模块测试，针对软件设计中的最小单位——程序模块，进行正确性检查的测试工作。

单元测试需要从程序的内部结构出发设计测试用例。多个模块可以平行地独立进行单元测试。

8.2 单元测试主要测试哪些问题?

1. **模块接口测试**
2. **局部数据结构测试**
3. **路径测试**
4. **错误处理测试**

5. **边界测试**

8.3 什么是插桩程序设计？

定义：
一种基本的动态测试方法，向源程序中添加一些语句实现对程序代码的执行、变量的变化等情况的检查，可以获得程序的控制流和数据流信息。

最简单的插装：
在程序中插入打印语句 `printf("...")` 语句。

插桩位置：
a. 程序的第一条语句
b. 分支语句的开始
c. 循环语句的开始
d. 下一个入口语句之前的语句
e. 程序的结束语句
f. 分支语句的结束
g. 循环语句的结束

插桩策略：

1. **语句覆盖探针（基本块探针）**：在基本块的入口和出口处，分别植入相应的探针，以确定程序执行时该基本块是否被覆盖。
2. **分支覆盖探针**：c/c++语言中，分支由分支点确定。对于每个分支，在其开始处植入一个相应的探针，以确定程序执行时该分支是否被覆盖。
3. **条件覆盖探针**：c/c++语言中，if、switch、while、do-while、for几种语法结构都支持条件判定，在每个条件表达式的布尔表达式处植入探针，进行变量跟踪取值，以确定其被覆盖情况。

设计插桩程序需要注意的几点：

1. 探测那些信息
2. 在什么位置设置探针
3. 设置多少个探测点
4. 特定位置插入用以判断变量特性的语句

8.4 断言

测试代码中用来**验证实际结果是否符合预期结果**的判断语句。

```
断言 = 判断 + 报错
```

🔥 第9章 集成测试

9.1 什么是集成测试？

集成测试又叫组装测试，通常在单元测试的基础上，将所有程序模块进行有序的、递增的测试。重点测试不同模块的接口部分。

9.2 集成测试与单元测试、系统测试的区别是什么？

单元测试：

- 又称模块测试，针对软件设计中的最小单位——程序模块，进行正确性检查的测试工作。
- 测试最小单元
- 开发人员执行

集成测试：

- 测试模块间接口
- 关注组件交互
- 重点测试不同模块的接口部分

系统测试：

- 指的是将整个软件系统看为一个整体进行测试，包括对功能、性能、以及软件所运行的软硬件环境进行测试。
- 测试整体功能
- 端到端验证
- 目前系统测试主要由黑盒测试工程师在系统集成完毕后进行测试，前期主要测试系统的功能是否满足需求，后期主要测试系统运行的性能是否满足需求，以及系统在不同的软硬件环境中的兼容性等。

9.3 集成测试的方法有哪些？

集成测试有两种不同的方法：非增式测试和增式测试。

1. 非增式测试：

- 在配备辅助模块的条件下，对所有模块进行个别的单元测试。然后在此基础上，按程序结构图将各模块联接起来，把联接后的程序当作一个整体进行测试。
- 典型的测试方法有大爆炸集成测试。
- 非增式测试的做法是先分散测试，再集中起来一次完成集成测试。如果在模块的接口处存在错误，只会在最后的集成时一下子暴露出来，便于找出问题和修改。

2. 增式测试：

- 增式集成是逐步实现的，测试过程使用了较少的辅助模块，也就减少了辅助性测试工作。并且一些模块在逐步集成的测试中，得到了较为频繁的考验，因而可能取得较好的测试效果。
- 主要有两种实施顺序：
  - **自顶向下增式测试**：逐步集成和逐步测试是按结构图自上而下进行的。
  - **自底向上增式测试**：逐步集成和逐步测试是按结构图自下而上进行的。

⭐ 第10章 数据库测试

10.1 简述数据库应用软件与数据库管理系统软件的区别与联系。

区别：

- **数据库应用软件**：面向具体业务
- **DBMS（数据库管理系统）**：提供数据管理功能

联系：

- 应用软件基于DBMS
- 共同保证数据完整性
- 相互依赖运行

10.2 如何对数据库应用软件中设计的数据库模式的好坏进行验证？

1. ER图评审

2. 规范化程度检查
3. 业务规则验证
4. 性能评估
5. 数据一致性检查

10.3 如何测试DBMS的事务特性？

答案：

测试时，同时开启多个客户端连接，设置合适的隔离级别，按照设计的测试场景进行并发操作。

事务特性（ACID）：

- **原子性**：断电恢复测试
- **一致性**：并发更新测试
- **隔离性**：多事务并发访问
- **持久性**：故障恢复测试

隔离性测试示例：

```
用户1                           用户2
SQL> begin work;              SQL> begin work;
SQL> insert into test_1       SQL> insert into test_1
   values (3, 1);                values (3, 33);
SQL> select * from test_1     SQL> select * from test_1
   where a = 3;                   where a = 3;
A B                            A B
3 1                            3 1
SQL> commit;                   SQL> commit;
SQL> select * from test_1     SQL> select * from test_1
   where a = 3;                   where a = 3;
A B                            A B
3 1 33                         3 1 33
```

10.4 简述 MySQL 的 SQL 功能回归测试过程。

- 准备测试数据
- 编写测试用例
- 执行 SQL 语句
- 验证执行结果
- 对比历史结果
- 性能指标分析

10.5 请举例说明如何测试 DBMS 的事务特性。

- 原子性：断电恢复测试
- 一致性：并发更新测试
- 隔离性：多事务并发访问
- 持久性：故障恢复测试

10.6 如何对 DBMS 进行性能测试？性能测试中重点需要考虑哪些内容？

测试重点：

- 并发处理能力
- 响应时间
- 资源利用率
- 可扩展性

考虑内容：

- 数据量大小
- 并发用户数
- 查询复杂度
- 硬件配置

10.7 DBMS 的高可用性测试的主要挑战有哪些？

- 故障注入难度
- 环境复杂性
- 数据一致性验证
- 性能影响评估
- 恢复时间测试

⭐ 第11章 智能软件测试技术

11.1 简述第十一章所讲的几种智能搜索算法的优劣性。

- 深度优先：内存效率高，可能不是最优解
- 广度优先：保证最优解，内存消耗大
- A* 算法：效率与最优性平衡
- 启发式搜索：速度快，结果可能次优

11.2 简述遗传算法的过程。

- 初始种群生成
- 适应度评价
- 选择操作
- 交叉操作
- 变异操作
- 迭代进化

⭐ 第12章 公有云测试质量评估

12.1 云测试有哪两层含义?

测试云平台：

- 功能测试
- 性能测试
- 安全测试

云测试服务：

- 测试即服务
- 测试资源云化
- 测试过程管理

12.2 针对云软件质量属性定量化描述的问题，最直接的思路有哪两种?

- 质量模型建立
- 度量指标体系
- 权重确定方法
- 综合评估模型

12.3 请列举几种云安全技术。

- 数据加密
- 访问控制
- 身份认证
- 漏洞扫描
- 入侵检测

12.4 云计算安全与传统软件安全有哪些区别?

- 多租户安全
- 数据分布式存储
- 虚拟化安全
- 服务可用性要求
- 合规性要求

📖 其他重要知识点

13.1 什么是回归测试?

回归测试是指软件被修改后重新进行的测试，如重复执行上一个版本测试时的用例，是为了保证对软件所做的修改没有引入新的错误而重复进行的测试。

13.2 什么是软件质量保证（SQA）？

概念：
软件质量保证是贯穿软件项目整个生命周期的有计划和有系统的活动，经常针对整个项目质量计划执行情况进行评估，检查和改进，向管理者、顾客或其他方提供信任，确保项目质量与计划保持一致。

目的：
确保软件项目的过程遵循了对应的标准及规范要求且产生了合适的文档和精确反映项目情况的报告，其目的是通过评价项目质量建立项目达到质量要求的信心。

活动：
软件质量保证活动主要包括评审项目过程、审计软件产品，就软件项目是否真正遵循已经制定的计划、标准和规程等，给管理者提供可视性项目和产品可视化的管理报告。

13.3 什么是第三方测试?

定义：
第三方测试是由开发者和用户以外的第三方进行的软件测试，其目的是为了保证测试的客观性。

- 独立于开发方和用户方
- 专业测试机构
- 提供客观评估
- 保证测试公正性

意义：

- 保证测试独立性
- 提供专业服务
- 降低测试成本

模式：

- 完全外包
- 部分外包
- 现场服务

测试范围：

- 功能测试
- 性能测试
- 安全测试
- 兼容性测试
- 用户体验测试

13.4 请简述第三方测试的测试过程。

- 需求分析
- 测试方案制定
- 测试环境搭建
- 测试执行
- 问题跟踪
- 报告交付

13.5 第三方测试的实施主体和定义。

- 专业测试公司
- 认证机构
- 独立测试实验室
- 咨询服务机构

13.6 请简述第三方测试的核心内容。

- 测试策略制定
- 测试用例设计
- 测试执行管理
- 缺陷管理
- 质量评估
- 改进建议

13.7 什么是测试驱动开发（TDD）？

核心理念:

- 先写测试后写代码
- 小步快跑迭代
- 持续重构改进
- 测试驱动设计

优势:

- 提高代码质量
- 便于重构
- 文档作用

13.8 什么是测试左移？为什么要进行测试左移？

原因:

- 降低修复成本
- 提前发现问题
- 缩短反馈周期

实践:

- 需求阶段介入
- 设计阶段评审
- 编码阶段单测
- 持续集成验证

13.9 软件测试对工作人员有什么要求？

答案:

1. 技术能力
2. 业务理解能力
3. 沟通协作能力
4. 问题分析能力
5. 文档编写能力

13.10 如何进行API接口测试？

答案:

功能性:

- 接口规范性
- 返回值正确性
- 异常处理

非功能性:

- 性能响应
- 安全性
- 可靠性

覆盖维度:

- 参数验证
- 业务流程
- 边界条件

13.11 安全测试包括哪些主要的挑战？

- 攻击方式多样性
- 漏洞发现难度
- 测试环境风险
- 自动化程度低
- 专业人才缺乏

13.12 简述 SOFIA 检测 SQL 注入的过程。

- 静态代码分析
- 识别注入点
- 生成测试用例
- 执行注入测试
- 结果分析报告

13.13 企业的测试策略体现在几个方面？

- 测试目标设定
- 资源配置策略
- 测试流程规范
- 工具选型标准
- 质量控制方法

13.14 为什么要制订测试计划？

- 明确测试目标
- 合理分配资源

- 控制测试进度
- 规范测试过程
- 风险预防管理

13.15 简述基于 CMMI 的测试流程和传统测试流程的区别。

CMMI 特点：

- 过程文档化
- 度量和控制
- 持续改进
- 标准化程度高

传统测试：

- 灵活性强
- 文档较少
- 过程简单

13.16 了解当前互联网公司是如何将 DevOps 部署到企业的软件质量保障流程中的。

- 持续集成/交付
- 自动化测试
- 监控告警
- 快速反馈
- 协作文化

13.17 什么是测试用例的可维护性？如何提高测试用例的可维护性？

- 模块化设计
- 参数化处理
- 规范化命名
- 版本控制
- 文档完善

13.18 探索性测试和脚本化测试各有什么优缺点？

探索性测试优点：

- 灵活性高
- 快速发现问题
- 适合新功能

缺点：

- 依赖测试经验
- 难以重现

脚本化测试优点：

- 可重复执行
- 结果可对比
- 自动化程度高

缺点：

- 维护成本高
- 开发周期长

13.19 如何进行移动应用的兼容性测试？

硬件兼容性：

- 不同屏幕尺寸
- CPU/内存配置
- 传感器支持

软件兼容性：

- 操作系统版本
- 系统定制化 ROM
- 第三方应用冲突

13.20 什么是 A/B 测试？在什么场景下使用？

应用场景：

- 用户界面优化
- 功能特性验证
- 营销策略测试

关键要素：

- 对照组设计
- 样本量确定
- 数据收集分析
- 统计显著性

13.21 性能测试中的并发用户数和虚拟用户数有什么区别？

并发用户：

- 同时在线用户
- 实际业务场景

虚拟用户：

- 模拟测试负载
- 可控制行为

区别：

- 资源消耗
- 行为模式
- 统计方式

13.22 请解释测试策略和测试计划的区别。

测试策略：

- 长期指导方针
- 整体测试方法

- 框架性文档

测试计划：

- 具体执行方案
- 详细任务分解
- 资源时间安排

13.23 如何进行测试用例的优先级划分？

- 确定优先级划分的标准
- 收集相关信息
- 评估每个测试用例
- 划分优先级
- 记录和跟踪
- 定期审查和调整

📚 附录：计算题示例

示例1：基本路径测试

题目：

```java
int findMax(int a, int b, int c) {
    int max = a;
    if (b > max) {
        max = b;
    }
    if (c > max) {
        max = c;
    }
    return max;
}
```

要求： 画出控制流图，计算圈复杂度，写出独立路径，测试用例

答案：

（1）控制流图：

```
    2
   ↙ ↘
  3   6
   ↘   ↙
    4
   ↙ ↘
  5   6
   ↘   ↙
    7
```

（2）圈复杂度：
V(G) = 边数 - 节点数 + 2 = 7 - 6 + 2 = 3

（3）独立路径：

1. 1 → 2 → 3 → 6 → 7
2. 1 → 2 → 3 → 4 → 6 → 7
3. 1 → 2 → 3 → 4 → 5 → 6 → 7

（4）测试用例：

1. a=5, b=3, c=2（期望输出：5）
2. a=3, b=5, c=4（期望输出：5）
3. a=3, b=5, c=7（期望输出：7）

示例2：等价类划分

题目：
用户名注册系统，使用等价类划分写出测试用例：

1. 用户名长度为6到20个字符
2. 用户名开头必须为字母
3. 用户名只能含字母、数字和下划线
4. 用户名不能含有特殊字符

答案：

（1）有效等价类：

1. 长度在6-20之间的合法用户名
2. 以字母开头
3. 仅包含字母、数字和下划线

（2）无效等价类：

1. 长度小于6的用户名
2. 长度大于20的用户名
3. 不以字母开头的用户名
4. 包含特殊字符的用户名

（3）测试用例：

1. "abc123"（有效）
2. "abcdef_123456789"（有效）
3. "ab"（无效：长度过短）
4. "abcdefghijklmnopqrst1"（无效：长度过长）
5. "123abc"（无效：不以字母开头）
6. "abc@123"（无效：包含特殊字符）

示例3：条件覆盖和分支覆盖

题目：
保险金额计算，写出测试用例，要求满足条件覆盖和分支覆盖

```java
int insuranceCost(int age, boolean isSmoker, boolean hasDisease) {
```

```
    int p = 1000;
    if (age > 60 && isSmoker) {
        p *= 2;
    } else if (age > 30 && hasDisease) {
        p *= 1.5;
    } else if (age <= 30) {
        p *= 0.8;
    }
    return p;
}
```

答案：

（1）条件覆盖测试用例：

1. age=65, isSmoker=true, hasDisease=false（premium=2000.0）
2. age=35, isSmoker=false, hasDisease=true（premium=1500.0）
3. age=25, isSmoker=false, hasDisease=false（premium=800.0）
4. age=45, isSmoker=false, hasDisease=false（premium=1000.0）

（2）分支覆盖测试用例：

1. age=65, isSmoker=true, hasDisease=false（premium=2000.0）
2. age=35, isSmoker=false, hasDisease=true（premium=1500.0）
3. age=25, isSmoker=false, hasDisease=false（premium=800.0）

祝考试顺利！加油！💪